

Unit III

Exception Handling & Multithreading

Exception handling: Concepts and benefits of exception handling, exception hierarchy, usage of try, catch, throw, throws and finally, built-in and User Defined Exceptions

Multithreading: Definition thread, thread life cycle, creating threads, synchronizing threads, daemon threads

Control Flow In Exceptions– JVM reaction to Exceptions– Inter Communication of Threads– Critical Factor in Thread–Deadlock

* Exception Handling :-

The exception handling is one of the powerful Mechanism to handle the runtime errors so that normal flow of the application can be maintained.

* Exception :-

In general, Exception is an abnormal condition. In java, exception is an event that disrupts the normal flow of the program. (or)

An exception is a problem that arises during the execution of a program. When an Exception occurs the normal flow of the program is disrupted and The program terminates abnormally.

An exception can occur for many different reasons. The following are some scenarios where an exception occurs.

- A user has entered an invalid data.
- A file that needs to be opened cannot be found.
- A network connection has been lost in the middle of communication.

Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner.

Based on these, the exceptions are categories into three types, those are

- checked exceptions
- unchecked exceptions
- Errors.

There are mainly two types of exceptions: checked and unchecked where error is considered as unchecked exception.

→ Checked exceptions:

A checked exception is an exception that occurs at the compile time, these are also called as compile time exceptions. These exceptions cannot simply be ignored at the time of compilation, the programmer should take care of these exceptions.

Examples:

FileNotFoundException

ClassNotFoundException

NoSuchFieldException

EOFException, etc.

The checked exceptions can be ^(checked) handle at the time of compilation.

→ Unchecked exceptions:

An unchecked exceptions is an exception that occurs at the time of execution. These are also called as Runtime exceptions. These include programming bugs, such as logic errors or improper use of API. Runtime exceptions are ignored at the time of compilation.

Examples:

ArithmeticException

ArrayIndexOutOfBoundsException

NullPointerException

NegativeArraySizeException, etc.

The unchecked exceptions can be checked at runtime.

→ Errors:

These are not exceptions at all, but problems that arise beyond the control of the user or the programmer. Error defines problems that are not excepted by programmer or program.

Examples: Memory Error,
Hardware error, JVM error etc.

→ An exception is nothing but runtime error. It can be handled to provide a suitable message to user.

→ Java uses a mechanism or model to handle exceptions,

This mechanism is known as Exception Handling Mechanism.

→ Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `ArithmeticException`, `IO`, etc.

→ The core advantage of exception handling is to maintain the normal flow of the application. Normally exception interrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

```
statement 1;  
statement 2;  
statement 3;  
statement 4;  
statement 5; // exception occurs  
statement 6;  
statement 7;  
statement 8;  
statement 9;  
statement 10;
```

Suppose there is 10 statements in your program and there occurs an exception at statement 5, rest of code will not be executed i.e. statement 6 to 10 will not run.

If we perform exception handling, rest of the statements will be executed. That is why we use exception handling in Java.

→ There are 5 keywords used in Java exception handling.

- try
- finally
- throws
- catch
- throw

* Exception class :-

The exception class can handle any kind of exceptions by using built-in exception classes. There are many built-in classes in Exception class. Some of them are.

- ArithmeticException : Arithmetic error
- ArrayIndexOutOfBoundsException : Array index out of bounds Exception
- NullPointerException : Accessing an object through null pointer
- NumberFormatException : Given no. is not number
- ClassNotFoundException : class not found
- IOException : Input/output exception
- FileNotFoundException : unable to locate a file
- SQLException : sql statement is not well.

→ Exception Handling requires the following four steps

1. Finding the problem (Identify the statements whose execution may result in exception. put all those statements in a try {...} block.)
2. Inform that an exception is thrown (Throw the exception)
3. Receive the exception (Catch the exception using catch {...} block).
4. provide exception handling code in catch block.

* Try and catch block :-

• try block :

It is used to enclose the code that might throw an exception. i.e all statements that are likely to raise an exception at run time are kept in try block. This block will detect an exception and throws it. It will be handled by catch block.

Try block must be followed by either catch or finally block.

Syntax:- try - catch

```
try
{
    // code that may throw exception
}
catch (Exception_className ref)
{
    // statements
}
```

• catch block :

It is used to handle the exception. It must be used after the try block only. It provides a suitable message to the user, then user will be able to proceed with the application.

→ A try block can follow multiple catch blocks, i.e we can use multiple catch blocks with a single try.

→ try {} block may have one or multiple statements.

→ try {} block may throw a single type of exception or

multiple exceptions. But at a time it can throw only single type of exception.

Syntax:-

```
try
{
    statements that may throw exceptions
}
catch (ExceptionType 1 e1) { ... }
catch (ExceptionType 2 e2) { ... }
:
catch (ExceptionType n en) { ... }
```

Example :-

SampleException.java

```
class SampleException
{
    public static void main (String args [])
    {
        int a = 0;
        int b = 10;

        try
        {
            int c = b/a;
            System.out.println("The result is " + c);
        }
        catch (ArithmeticException e)
        {
            System.out.println("Divided by zero");
        }
        int sum = a + b;
        System.out.println("sum is " + sum);
    }
}
```

o/p:- javac SampleException.java

java SampleException

Divided by zero (or) java.lang.

sum is 10.

ArithmeticException:
1 error

* Multiple catch blocks :-

A try block can be followed by multiple catch blocks. you can have any number of catch blocks after a single try block.

If an exception occurs in try block then the exception is passed to the first catch block in the list. If the exception type matches with the first catch block it gets caught, if not the exception is passed down to next catch block.

Rule:- At a time only one Exception is occurred and at a time only one catch block is executed.

Rule: All catch blocks must be ordered from most specific to most general. i.e. catch for `ArithmeticException` must come before catch for `Exception`.

Example:-

MultipleException.java

```
class MultipleException
{
    public static void main(String args[])
    {
        try
        {
            int arr[] = {1, 2};
            arr[4] = 3/0;
        }
        catch (ArithmeticException ae)
        {
            System.out.println("Divide by Zero");
        }
        catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println("array index out of bound");
        }
    }
}
```

O/P:- `javac MultipleException.java`
`java MultipleException`
Divide by zero.

Example - for unreachable catch block

While using multiple catch statements, it is important to remember that all catch blocks must be ordered from most specific to most general. i.e we have to write general exception class as last catch block.

URException.java

```
class URException
{
    public static void main (String args[])
    {
        try
        {
            int a[] = {1, 2};
            a[3] = 5/0;
        }
        catch (Exception e) // This block handles all exceptions
        {
            System.out.println("Generic Exception");
        }
        catch (ArithmeticException ae) // This block is unreachable
        {
            System.out.println("Divided by zero");
        }
    }
}
```

output:- javac URException.java
Compile Time Error

The above program arises compile time error, because you write general exception catch block as first catch block. so we need to write most specific exceptions first and then general exception catch block.

The general Exception class handles all exceptions if no other specific exception catch blocks.

* finally block:-

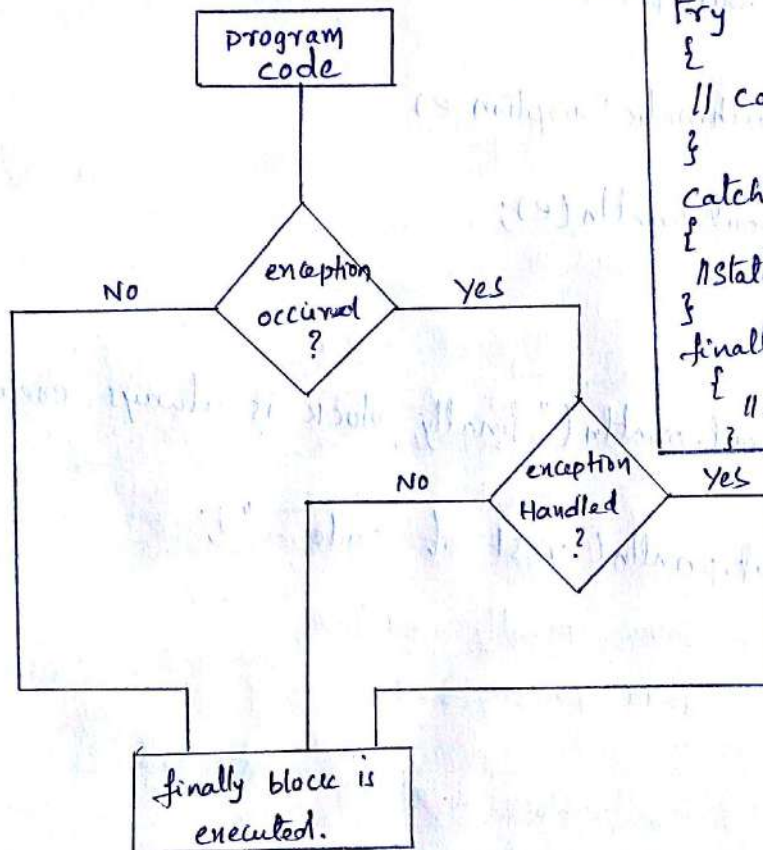
The finally block is used to execute important code such as closing connection, closing a file etc.

finally block is always executed whether exception is handled or not.

The finally block follows try or catch block.

Syntax:-

```
try
{
    // code
}
catch (exceptionType e)
{
    // statements
}
finally
{
    // statements
}
```



Note:- If you don't handle exception, before terminating the program JVM executes finally block (if any).

→ Finally block in Java can be used to put "cleanup" code such as closing a file, closing connection etc.

→ Let's see the different cases where finally block can be used/ executed.

The finally block can be executed ^{when/} where exception doesn't occur, exception occurs and not handled, exception occurs and handled.

Rule:- For each try block there can be zero or more catch blocks, but only one finally block.

Example:- Where exception doesn't occur
FinallyCase1.java

```
class FinallyCase1
{
    public static void main(String args[])
    {
        try
        {
            int data = 25/5;
            System.out.println(data);
        }
        catch (ArithmeticException e)
        {
            System.out.println(e);
        }
        finally
        {
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of code...");
    }
}

//p: javac FinallyCase1.java
// java FinallyCase1
// 5
// finally block is always executed
// rest of code...
```

Example Where exception occurs and not handled
FinallyCase2.java

```
class FinallyCase2
{
    public static void main(String args[])
    {
        try
        {
            int data = 25/0;
            System.out.println(data);
        }
    }
}
```

catch (NullPointerException e)

```
{
    System.out.println(e);
}
```

finally

```
{
    System.out.println("finally block is always executed");
}
```

```
System.out.println("rest of the code ...");
```

```
}
```

```
}
```

O/P:- javac FinallyCase2.java

java FinallyCase2

finally block is always executed

Exception in thread "Main" java.lang.ArithmeticE

xception: / by zero

Example: Where exception occurs and handled.

FinallyCase3.java

```
class FinallyCase3
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        try
```

```
        {
```

```
            int data = 25/0;
```

```
            System.out.println(data);
```

```
        }
```

```
        catch (ArithmeticException e)
```

```
        {
```

```
            System.out.println("Divided by zero");
```

```
        }
```

```
        finally
```

```
        {
            System.out.println("finally block is always executed");
        }
```

```
        System.out.println("rest of the code ...");
```

```
    }
```

```
}
```

O/P:- javac FinallyCase3.java

java FinallyCase3

Divided by zero

finally block is always executed

rest of code ...

* throw Keyword :-

The java throw keyword is used to explicitly throw an exception. We can throw either checked or unchecked exception in java by throw keyword.

program execution stops on encountering throw statement.

Syntax:- throw throwableinstance
(or)
throw exception

Example:-

In this example, we have created the validate method that takes integer value as parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message "Welcome to vote".

ThrowExp.java

```
class ThrowExp
{
    void validate (int age)
    {
        if (age < 18)
            throw new ArithmeticException ("not valid");
        else
            System.out.println ("Welcome to vote");
    }
    public static void main (String args[])
    {
        ThrowExp t = new ThrowExp();
        t.validate (13);
        System.out.println ("rest of code");
    }
}
```

o/p: javac ThrowExp.java
java ThrowExp
java.lang.ArithmeticException: not valid

* Throws keyword :-

The throws keyword is used to declare an exception. It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

The throws keyword mainly applied on methods to provide information to caller of the method about the exception.

Syntax:-

```
return-type method_name() throws exception_class_name  
{  
    //method code  
}
```

Example:-

Let's see the example of java throws keyword which describes that exceptions.

```
public class ThrowsDemo {  
    void validate (int age) throws ArithmeticException  
    {  
        if (age < 18)  
            throw new ArithmeticException ("not valid");  
        else  
            System.out.println ("Welcome to vote");  
    }  
}
```

```
public static void main (String args[])
```

```
{  
    try  
    {  
        ThrowsDemo td = new ThrowsDemo();  
        td.validate(12);  
    }  
    catch (ArithmeticException e)
```

```
{  
        System.out.println(e);  
    }  
    System.out.println("rest of code.");  
}
```

o/p: javac ThrowsDemo.java
java ThrowsDemo
java.lang.ArithmeticException
: not valid.
rest of code..

* Multithreading:-

* Introduction:-

First, we need to know about Multitasking.

→ Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Example:- class room student
1. Lising
2. writing

→ Multitasking can be achieved by two ways (or) Multitasking is classified into two types

- process-Based Multitasking (Multiprocessing)
- Thread - Based Multitasking (Multithreading)

* process based Multitasking:-

Executing multiple tasks simultaneously, where each task is separate independent process (or) program is called as process based Multitasking.

Example:-

1. Typing a java program in notepad.
2. Listen audio songs.
3. Download a file from internet.

The above three tasks are performed simultaneously in a system, but there is no dependence between one task & another task.

→ process based Multitasking is best suitable at "operating system" level not at programming level.

* Thread - based Multitasking:-

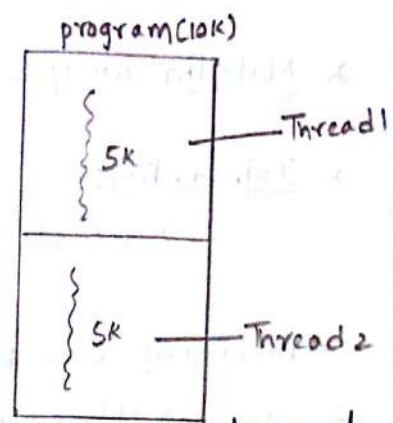
Executing multiple tasks simultaneously, where each task is a separate independent part of the same program (or) process is called Thread-based Multitasking.

→ The each independent part is called a thread.

→ Thread-based Multitasking is best suitable at programming level.

Example:-

Let a program has 10K lines of code, where last 5K lines of code doesn't depend on first 5K lines of code. Then both are started the execution simultaneously. So it takes less time to complete execution.

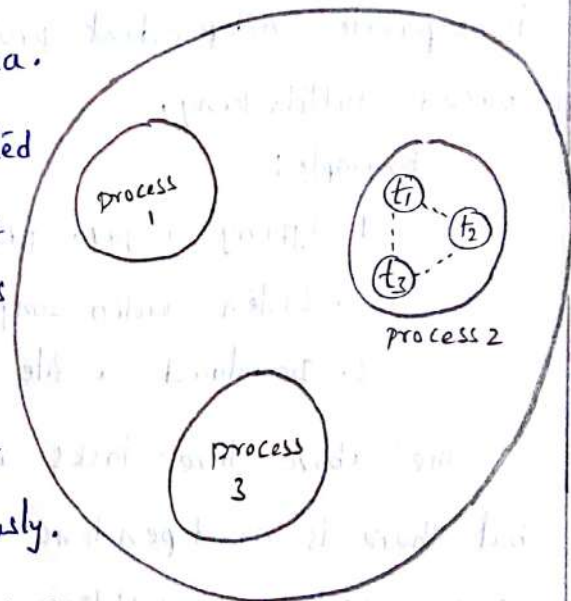


Note:- Any type of Multitasking is used to reduce response time of System and improves performance.

* MultiThreading:-

- A Thread is a lightweight sub process, a smallest unit of processing. It is a separate path of execution.
- Threads are independent, if there occurs exception in one thread, it doesn't affect other threads.
- It shares a common memory area.

→ As shown in figure, Thread is executed inside the process. There can be Multiple process inside the O.S and one process can have multiple threads.



Defⁿ:- MultiThreading is a process of executing multiple threads simultaneously.

↳ Multiprocessing and MultiThreading, both are used to achieve multitasking.

fig: operating System.

But we use multiThreading than multiprocessing because threads share a common memory area and context-switching between threads takes less time than process.

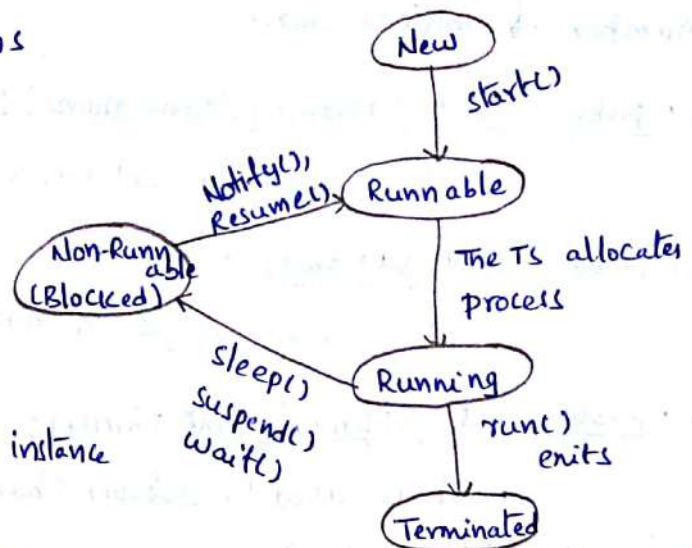
↳ The main application areas of MultiThreading are

- To develop Multi-media movies
- To develop video games
- To develop web servers & Application servers, etc.

* Life Cycle of a Thread (or) A Thread Model :-

The Life cycle of the Thread is controlled by JVM. And it has five states as follows

- New
- Runnable
- Running
- Non-Runnable (blocked)
- Terminated



* New :- In this state, the new instance of Thread class is created.

Eg:- `MyThread t = new MyThread();`

Fig:- Life cycle of a Thread.

* Runnable :- In this, the thread is ready to execute after invocation of `start()` method. Eg:- `t.start();`

* Running :- In this, the thread is running by using `run()` method.

* Non-Runnable (Blocked) :-

In this, the thread is in blocked state, i.e the thread is still alive, but is currently not eligible to run.

* Terminated (Dead) :-

In this, the thread is in dead state, when `run()` method exits (or) completed the process.

* Thread class :-

The Thread class provides methods and constructors to create and perform operations on a thread.

↳ commonly used methods of Thread class :-

1. public void run() : It is used to perform action for a thread.
2. public void start() : It is used to start the execution of the thread. JVM calls the `run()` method on the thread.

3. public void sleep (long milliseconds):
To stop the execution of thread for the specified number of milliseconds.
4. public void setName (String name):
It is used to set (or) changes the name of a thread.
5. public void getName():
It is used to get the name of a thread.
6. public int setPriority (int priority):
It is used to set (or) change the priority of the thread.
7. public int getPriority():
It returns the priority of the thread.
8. public boolean isAlive():
It checks if the thread is alive or not.
9. public void yield():
It is used to pause the currently executing thread and allow other threads to execute.
10. public void suspend():
It is used to suspend the thread.
11. public void resume():
It is used to resume the suspended thread.
12. public void stop():
It is used to stop the execution of thread.

↳ Constructors:

1. Thread() - Without parameters/arguments.
2. Thread(String name) - With one string argument.
3. Thread(Runnable r) - With runnable argument.
4. Thread(Runnable r, String name) - With runnable and string arguments.

* creating Thread :-

There are two ways to create a thread

- ↳ By extending Thread class.
- ↳ By implementing Runnable interface.

↳ By extending Thread class :-

In This, By extending Thread class we can able to create a thread and able to start a thread by calling start() method of Thread class.

Syntax:-

```
class class-name extends Thread
{
    ...
}
```

In thread class, we can mainly use two methods run() and start().

Example:-

```
class A extends Thread
{
    public void run()
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println("Thread A value = "+i);
        }
    }
}
```

```
class SimpleThread
{
    public static void main (String arg[])
    {
        A a = new A();
        a.start();
    }
}
```

output: javac SimpleThread.java
java SimpleThread
Thread A value = 1
" 2
" 3
" 4
" 5

↳ By implementing Runnable interface:-

In this, By implementing runnable interface, we can able to create & execute thread.

→ The Runnable interface should be implemented by any class.

→ Runnable interface have only one Thread method named run(), where run() method is abstract method.

→ The programmer should declare object for sub class, that object can be used as argument in thread class constructor from Thread object.

Syntax:- class class-name implements Runnable

```
{  
    public void run()  
    {  
        // ...  
    }  
}
```

ThreadRunDemo.java

Example:-

```
class A implements Runnable  
{  
    public void run()  
    {  
        for(int i=1; i<=5; i++)  
        {  
            System.out.println("Thread A value = " + i);  
        }  
    }  
}
```

```
class ThreadRunDemo  
{  
    public static void main(String a[])  
    {  
        A a = new A();  
        Thread t = new Thread(a);  
        t.start();  
    }  
}
```

output:- javac ThreadRunDemo.java
java ThreadRunDemo
Thread A value = 1
 2
 3
 4
 5

* Thread priority:-

→ In java, each thread has a priority. priorities are represented by a number between 1 and 10.

→ In most cases, thread scheduler schedules the threads according to their priority.

→ But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

→ There are three constant thread priorities defined in Thread class.

1. public static int MIN_PRIORITY
2. public static int NORM_PRIORITY
3. public static int MAX_PRIORITY

→ The value of MIN_PRIORITY is 1,

The value of NORM_PRIORITY is 5,

and The value of MAX_PRIORITY is 10.

Example:

```
class Thread1 extends Thread
{
    public void run()
    {
        for (int i=1; i<=5; i++)
        {
            System.out.println("Thread1 i=" + i);
        }
        System.out.println("Thread1 is in terminated state");
    }
}

class Thread2 extends Thread
{
    public void run()
    {

```

```
for (int j=1; j<=5; j++)
```

```
{
```

```
System.out.println("Thread2 j=" + j);
```

```
}
```

```
System.out.println("Thread2 is in terminated state");
```

```
}
```

```
}
```

```
class Thread3 extends Thread
```

```
{
```

```
public void run()
```

```
{
```

```
for (int k=1; k<=5; k++)
```

```
{
```

```
System.out.println("Thread3 k=" + k);
```

```
}
```

```
System.out.println("Thread3 is in terminated state");
```

```
}
```

```
}
```

```
class ThreadPriorityDemo
```

```
{
```

```
public static void main (String args[])
```

```
{
```

```
Thread1 a = new Thread1();
```

```
Thread2 b = new Thread2();
```

```
Thread3 c = new Thread3();
```

```
System.out.println("Default priority for Thread1 is:" + a.getPriority());
```

```
System.out.println("Default priority for Thread2 is:" + b.getPriority());
```

```
System.out.println("Default priority for Thread3 is:" + c.getPriority());
```

```
a.setPriority(Thread.MIN_PRIORITY);
```

```
b.setPriority(Thread.NORM_PRIORITY);
```

```
c.setPriority(Thread.MAX_PRIORITY);
```

```

System.out.println("New priority of Thread1 is: " + a.getPriority());
System.out.println("New priority of Thread2 is: " + b.getPriority());
System.out.println("New priority of Thread3 is: " + c.getPriority());

```

```

a.start();
b.start();
c.start();
}
}

```

output: javac ThreadpriorityDemo.java

java ThreadpriorityDemo

Default priority of Thread1 is: 5

" " Thread2 is: 5

" " Thread3 is: 5

New priority of Thread1 is: 1

" " Thread2 is: 6

" " Thread3 is: 10

Thread3 k=1

" k=2

" k=3

" k=4

" k=5

Thread3 is in terminated state

Thread2 j=1

" j=2

" j=3

" j=4

" j=5

Thread2 is in terminated state

Thread1 i=1

" i=2

" i=3

" i=4

" i=5

Thread1 is in terminated state

* Methods of Thread class:-

→ sleep():-

The sleep() method of Thread class is used to stop the execution of thread for the specified amount of time.

Syntax: sleep(long milliseconds)

The sleep() may throws an "InterruptedException". i.e we need to use try & catch statements while using sleep() method.

Example:-

```
class sleepMethod extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        for (int i=1; i<=5; i++)
```

```
        {
```

```
            try
```

```
            {
```

```
                Thread.sleep(500);
```

```
            }
```

```
        catch (InterruptedException e)
```

```
        {
```

```
            System.out.println(e);
```

```
        }
```

```
        System.out.println(i);
```

```
    }
```

```
    }
```

```
}
```

```
class ThreadSleepDemo
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        SleepMethod t1 = new SleepMethod();
```

```
        SleepMethod t2 = new SleepMethod();
```

```
        t1.start();
```

```
        t2.start();
```

```
    }
```

```
}
```

output:- javac ThreadSleepDemo.java

java ThreadSleepDemo

1

1

2

2

3

3

4

4

5

5

Note:- If you sleep a thread for the specified time, the thread scheduler picks up another thread and so on.

→ Naming Thread:-

→ The Thread class provides methods to change and get the name of a thread, those are setName() and getName() methods.
→ By default, each thread has a name i.e thread-0, Thread-1 and so on.

getName(): It is used to get the name of thread.

Syntax: getName()

setName(): It is used to set (or) change the name of thread

Syntax: setName(String name)

Example:

```
class A extends Thread
{
    public void run()
    {
        System.out.println("Thread A is running...");
    }
}

class B extends Thread
{
    public void run()
    {
        System.out.println("Thread B is running...");
    }
}

class ThreadNameDemo
{
    public static void main(String args[])
    {
        A a = new A();
        B b = new B();
        System.out.println("Default Name of Thread A is"
                           + a.getName());
        System.out.println("Default Name of Thread B is"
                           + b.getName());
        a.setName("Madhu");
        b.setName("Hani");
    }
}
```

```
System.out.println("New Name of Thread A is" + a.getName());
```

```
System.out.println("New Name of Thread B is" + b.getName());
```

```
a.start();
```

```
b.start();
```

```
}  
}
```

Output: java ThreadNameDemo.java

java ThreadNameDemo

Default Name of Thread A is Thread-0

" " B is Thread-1

New Name of Thread A is Madhu

" " B is Hari

Thread A is running

Thread B is running

→ Joining Threads:-

Sometimes one thread need to know when other thread is terminating (or) not.

In Java, `isAlive()` and `join()` are two different methods that are used to check whether a thread has finished its execution or not.

* `isAlive()`: This method returns true if the thread still running otherwise it returns false.

Syntax: `isAlive()`

Example:

ThreadIsAliveDemo.java

```
class MyThread extends Thread  
{  
    public void run()  
    {  
        System.out.println("r1");  
        try  
        {  
            Thread.sleep(500);  
        }  
        catch (InterruptedException e)  
        {  
            System.out.println(e);  
        }  
    }  
} System.out.println("r2");
```

A)

```

class ThreadIsAliveDemo
{
    public static void main(String arg[])
    {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        // t2.start();
        System.out.println(t1.isAlive());
        System.out.println(t2.isAlive());
    }
}

```

output: javac ThreadIsAliveDemo.java
 java ThreadIsAliveDemo
 true
 false
 r1
 r2

* join():- This method waits for a specified thread completes its execution. It allows us to specify the time for which you want to wait for the specified thread to terminate.

Syntax: join()
 (or)
 join(long milliseconds)

→ join() method throw InterruptedException, i.e we need to use try & catch statements while using join() method.

Example:

```

class Thread1 extends Thread
{
    public void run()
    {
        for(int i=1; i<=5; i++)
        {
            try
            {
                Thread.sleep(500);
            }
        }
    }
}

```

```

        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Thread1 value is: " + i);
    }
}
}
class Thread2 extends Thread
{
    public void run()
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println("Thread2 value is: " + i);
        }
    }
}

```

```

class ThreadJoinDemo
{
    public static void main(String args[])
    {
        Thread t1 = new Thread1();
        Thread t2 = new Thread2();
        t1.start();
        try
        {
            t1.join();
        }
        catch (InterruptedException e) { }
        t2.start();
    }
}

```

output:- javac ThreadJoinDemo.java
java ThreadJoinDemo

Thread1 value is: 1

" " 2
" " 3
" " 4
" " 5

Thread2 value is: 1

" " 2
" " 3
" " 4
" " 5

Example :- Example for Multithreading, Race between Hare & Tortoise story. (8)

```
class Tortoise extends Thread
```

1

```
public void run()
```

३

```
for (int i=1; i<=101; i++)
```

}

```
{
    System.out.println("Distance covered by Tortoise = " + i);
}
```

3

```
System.out.println("Tortoise has completed the Race...");
```

3

3

```
class Hare extends Thread
```

{

```
public void run()
```

3

```
for(int j=1; j<=101; j++)
```

}

```
System.out.println("Distance covered by Hare = " + j);
```

}

```
System.out.println("Hare has completed the race...");
```

3

3

class Race

{

```
public static void main (String args[])
```

3

```
Tortoise t = new Tortoise();
```

Have h = new Have(); output:- javac Race.java

```
t.start();
```

n. starting;

3

3

output:- javac Race.java

java Race

Distance covered by Tortoise = 1

• • • 2

Distance covered by Hare = 1

u . . 2

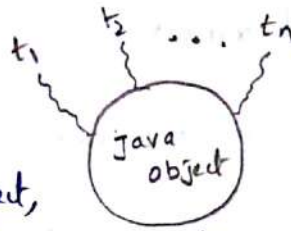
Tortoise has completed the Race..

Have has completed the Race..

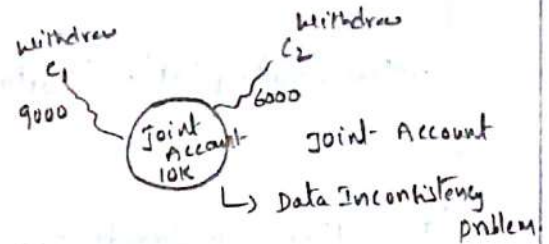
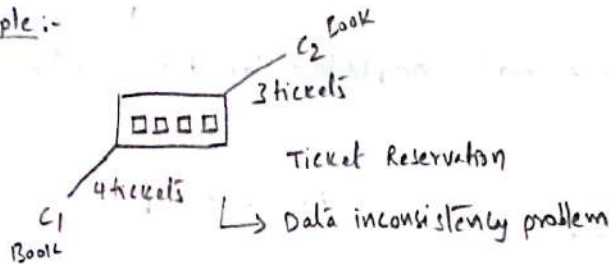
* Synchronization in threads :-

Synchronization is a process, that allows only one thread to access shared resource at a time, if multiple threads trying to access shared resource.

→ If multiple threads are trying to operate simultaneously on same java object, then there may be a chance of data inconsistency problem.



Example :-



→ To overcome this problem, we should go for Synchronized Keyword.

→ Synchronized is a modifier applicable only for methods and blocks but not for classes & variables.

→ If a method declared as Synchronized then at a time only one thread is allowed to execute that method on given object so that data inconsistency problem will be resolved.

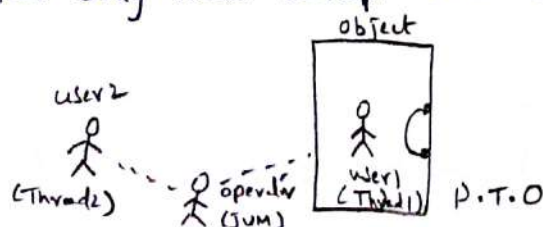
→ The main advantage of Synchronized Keyword is we can resolve data inconsistency problems, but the main disadvantage of Synchronized Keyword is it increases waiting time of threads and creates performance problems.



→ It increases response time if there are many users.

Note :- "Hence if there is no specific requirements, then it is not recommended to use Synchronized Keyword."

↳ Internally, Synchronization is implemented by using lock. Every object in java has an unique lock in java, whenever we are using Synchronized Keyword then only lock concept will come into the picture.

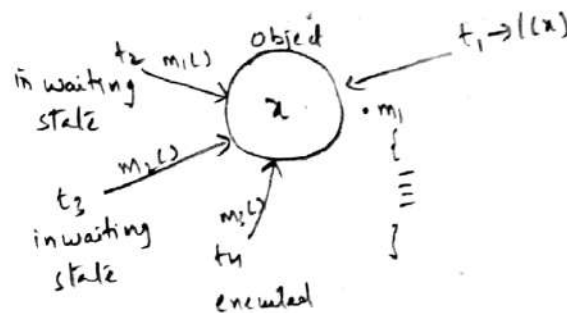
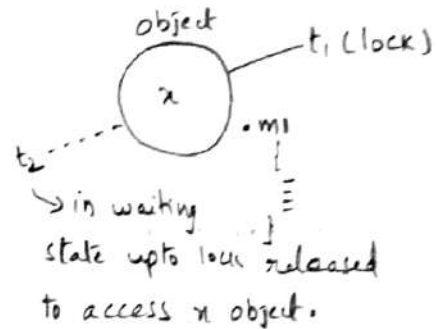


If a thread, wants to execute synchronized method on the given object, first it has to get lock of that object, one thread got the lock then it is allowed to execute any synchronized method on that object.
 → once method execution completes automatically thread releases the lock.

→ Acquiring & releasing lock internally takes care by JVM not by programmer (or) Thread.

Example

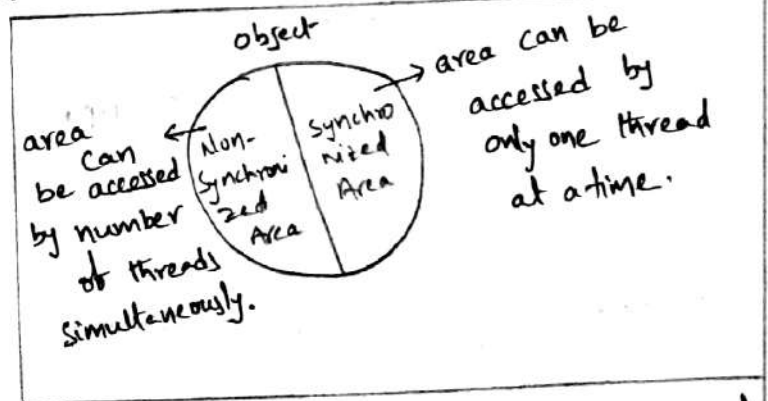
```
class X
{
    synchronized m1();
    synchronized m2();
    m3();
}
```



→ While a thread executing synchronized method on the given object, the remaining threads are not allowed to execute any synchronized methods simultaneously on the same object, but remaining threads allowed to execute non-synchronized methods simultaneously.

Note:

Lock concept is implemented based on object but not based on method.



→ In java, any object can have two areas, those are Synchronized area and non-synchronized area.

→ Synchronized area allows a thread to update (or) delete data.

→ Non-Synchronized area allows a thread to read the data (or) content.

* We should know, which method is declared as Synchronized (or) which method is declared as non-Synchronized method.

→ Where ever, we are performing update operations (add/remove/replace) that methods can be declared as Synchronized (or) declared in Synchronized area.

→ Where ever, we performing read operation, that methods can be declared as non-Synchronized / area.

Ex:- class x
{
 Synchronized Area
 {
 // object state changing
 }
 Non-Synchronized Area
 {
 // object state won't be changed
 }
}

Example class Banking
{
 Non-Synchronized Balance Enquiry()
 {
 // Just read operation
 }
 Synchronized withdraw()
 {
 // update
 }
}

Example class TicketReservation
{
 Non-Synchronized SeatAvailability()
 {
 // Just read operation
 }
 Synchronized Booking()
 {
 // update
 }
}

```
class Display
{
    public synchronized void wish(String name)
    {
        for (int i=1; i<=5; i++)
        {
            System.out.println("Good Morning:");
        }
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e) {}
        System.out.println(name);
    }
}

class MyThread extends Thread
{
    Display d;
    String name;
    MyThread(Display d, String name)
    {
        this.d = d;
        this.name = name;
    }
    public void run()
    {
        d.wish(name);
    }
}

class SynchronizedDemo
{
    public static void main(String args[])
    {
        Display d = new Display();
        MyThread t1 = new MyThread(d, "Dhoni");
    }
}
```

```
MyThread t2 = new MyThread(d, "yuvraj");
```

```
t1.start();
```

```
t2.start();
```

```
}  
}
```

output:- javac SynchronizedDemo.java

```
java SynchronizedDemo
```

```
Good Morning : Dhoni
```

```
" " : Dhoni
```

```
" " : Dhoni
```

```
" " : Dhoni
```

```
" " : Dhoni
```

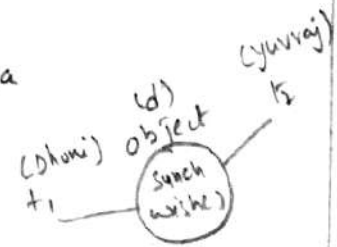
```
Good Morning : yuvraj
```

```
" " : yuvraj
```

```
" " : yuvraj
```

```
" " : yuvraj
```

```
" " : yuvraj
```



Example:-

TSynchronizationDemo.java

```
class printTable
```

```
{
```

```
public Synchronized void printTable(int n)
```

```
{  
System.out.println("Table of " + n);
```

```
for (int i=1; i<=10; i++)
```

```
{  
System.out.println(n * i);
```

```
try
```

```
{  
Thread.sleep(2000);
```

```
}  
catch (InterruptedException e) {}
```

```
}
```

```
}
```

```
}
```

```
class MyThread1 extends Thread
```

```
{
```

```
printTable pt;
```

```
MyThread1(printTable pt)
```

```
{
```

```
this.pt = pt;
```

```
}
```

```

    public void run()
    {
        pt.printTable(2);
    }
}
class MyThread2 extends Thread
{
    printTable pt;
    MyThread2(printTable pt)
    {
        this.pt = pt;
    }
    public void run()
    {
        pt.printTable(5);
    }
}
class TSynchronizationDemo
{
    public static void main(String args[])
    {
        printTable obj = new printTable();
        MyThread1 t1 = new MyThread1(obj);
        MyThread2 t2 = new MyThread2(obj);
        t1.start();
        t2.start();
    }
}

```

output:- javac TSynchronizationDemo.java

java TSynchronizationDemo

Table of 2

2
4
6
...

20

Table of 5

5
10
15
...

50

* Interthread communication :-

→ Interthread communication is a mechanism in which one thread is communicate with another thread by using `wait()`, `notify()` and `notifyAll()` methods.

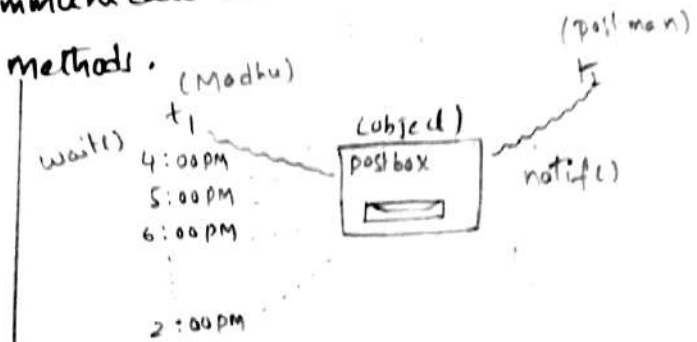
(or)

→ Interthread communication is all about allowing Synchronized threads to communicate with each other.

→ Two threads can communicate with each other by using `wait()`, `notify()` and `notifyAll()` methods.

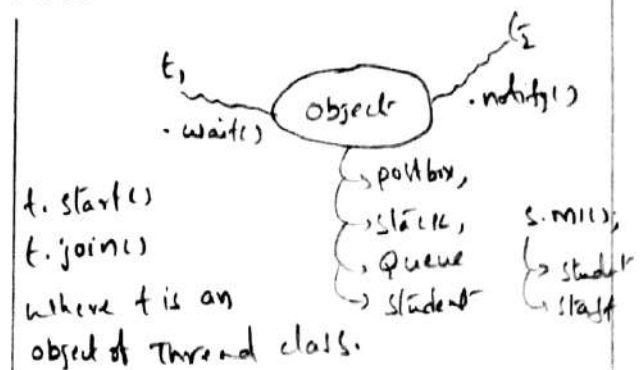
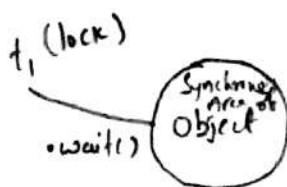
→ The thread, which is expecting updation is responsible to call `wait()` method, then immediately entered into waiting state

→ The thread, which is responsible to perform updation, after performing updation, it is responsible to call `notify()`, then waiting Thread will get that notification and continue its execution with those updated information.



In this, t_1 (Modhu) needs updation, so t_1 is responsible to call `wait()`, and t_2 (Postman) needs to perform updation, so t_2 is responsible to call `notify()`.

Note:- `wait()`, `notify()` and `notifyAll()` methods present in object class, but not in Thread class. Because a thread call `wait()`, `notify()` and `notifyAll()` on any object.



→ To call `wait()`, `notify()` (or) `notifyAll()` methods on any object, Thread should be owner of that object. i.e the Thread should has lock of that object, i.e the Thread should be inside Synchronized Area.

Hence, we can call `wait()`, `notify()` and `notifyAll()` methods from Synchronized Area otherwise we will get runtime Exception i.e "Illegal Monitor State Exception".

```

        public void run()
        {
            pt.printTable(2);
        }
    }
    class MyThread2 extends Thread
    {
        printTable pt;
        MyThread2(printTable pt)
        {
            this.pt = pt;
        }
        public void run()
        {
            pt.printTable(5);
        }
    }
    class TSynchronizationDemo
    {
        public static void main(String args[])
        {
            printTable obj = new printTable();
            MyThread1 t1 = new MyThread1(obj);
            MyThread2 t2 = new MyThread2(obj);
            t1.start();
            t2.start();
        }
    }

```

output :- javac TSynchronizationDemo.java
 java TSynchronizationDemo
 Table of 2
 2
 4
 6
 ...
 20
 Table of 5
 5
 10
 15
 ...
 50

②
→ If a thread calls `wait()` method of any object, it immediately releases lock of that particular object and entered into waiting state.

→ If a thread calls `notify()` of any object, it releases lock of that object, but may not immediately.

Note:- Except `wait()`, `notify()` & `notifyAll()`, there is no other method where thread releases the lock.

Methods:-

*wait()

→ `wait()` waits for until get notification (or) specified milliseconds.

Syntax:- `public final void wait()` throws `InterruptedException`

(or)

`public final void wait(long ms)` throws `InterruptedException`

notify()

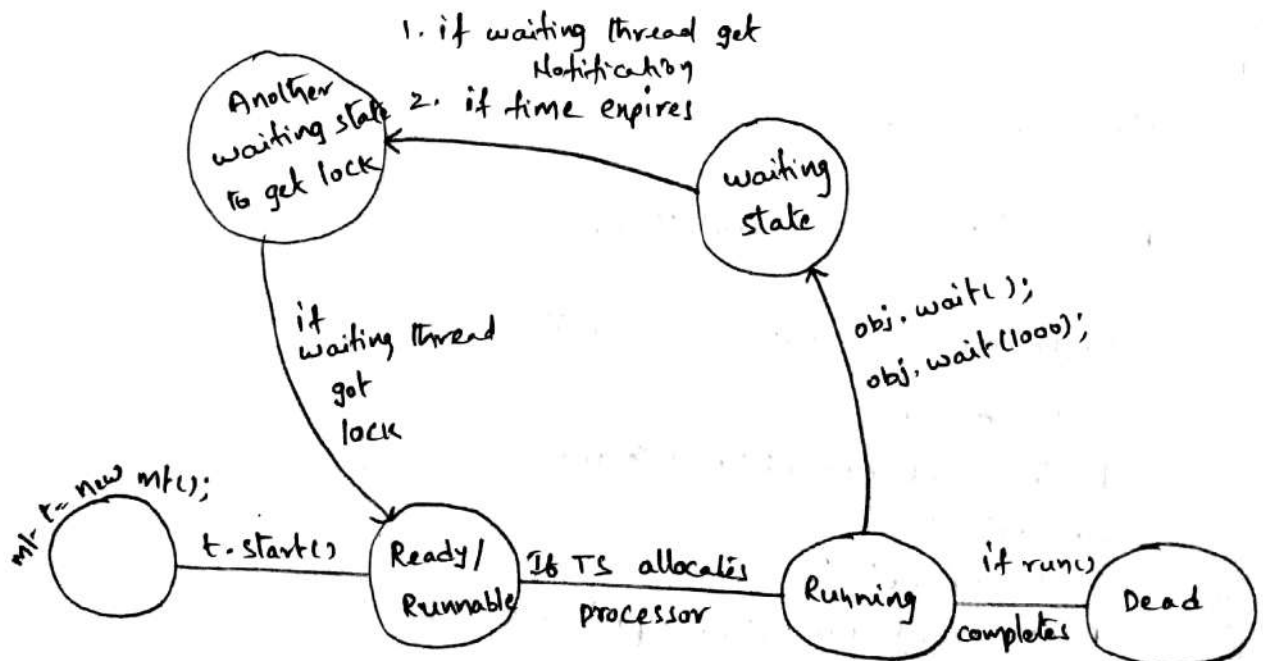
→ `notify()` gives notification to another thread

Syntax:- `public final void notify()`

(or)

`public final void notifyAll()`

→ understanding `wait()` & `notify()` methods in Thread lifecycle



Example:-

```
class Customer
{
    int amount=10000;
    Synchronized void withdraw (int amount)
    {
        System.out.println("going to withdraw...");
        if (this.amount < amount)
        {
            System.out.println("Less Balance ; waiting for deposit...");
            try
            {
                wait();
            }
            catch (Exception e) { }
        }
        this.amount -= amount;
        System.out.println("withdraw completed");
    }
    Synchronized void deposit (int amount)
    {
        System.out.println("going to deposit...");
        this.amount += amount;
        System.out.println("Deposit completed...");
        notify();
    }
}

class JTCDemo
{
    public static void main (String args[])
    {
        Customer c = new Customer();
        new Thread() {
            public void run() { c.withdraw (15000); }
        }.start();
        new Thread() {
            public void run () { c.deposit (10,000); }
        }.start();
    }
}
```

output:- javac JTCDemo.java
java JTCDemo
going to withdraw
Less Balance ; waiting for deposit
going to deposit
deposit completed
withdraw completed.